Security Document

Rotana Spa Management System

1. Security goals

- Protect member/patient data and company data from unauthorised access.
- Enforce least privilege via role-based access control.
- Keep a tamper-evident audit trail of all data changes.
- Prevent common web attacks (SQL injection, XSS, CSRF, brute force, IDOR/tenant cross-access).

2. User roles & permissions

The system ships five roles. Permissions are enforced **server-side** on every API route.

Role	Scope
`super_admin`	Platform-wide. Manages companies, demo licenses, audit logs, manuals. No tenant company of their own — cannot insert tenant rows without an explicit `company_id`.
`admin`	Company administrator — full access to one company's data, users, roles, settings.
`manager`	Operational access, view/edit/approve on business resources; reduced administrative rights.
`receptionist`	Front-desk workflows: check-ins, appointments, registrations, queue, towels, service orders. No therapist/service catalogue editing.
`guest`	Limited, view-only.

Permission model

- **Resource groups:** Dashboard, Customers, Membership, Operations, Gym, Spa, Inventory, Staff, Facilities, Reports, System & Administration.
- **Actions per resource:** `view`, `create`, `edit`, `delete`, `approve`.
- Stored in `role_permissions` (one row per role+resource).
- `lib/permissions.ts` (`requirePermission`, `can`) is used by every handler; pages also gate navigation by the same resource.

3. Authentication

- **Password:** bcrypt-hashed; plain text is never stored or logged.
- **Session:** JWT bearer token returned at login; sent as `Authorization: Bearer <token>`.

- **Entry points:** separate login handlers for admin, super-admin, employee and guest so roles are granted explicitly.
- **Deactivation:** inactive users are rejected at login and on token validation.
- **Recommendation:** force HTTPS in production and rotate `JWT_SECRET` on personnel change.

4. Authorisation & tenant isolation

- Every tenant table carries `company_id`; every query adds a `company_id` predicate.
- Ownership guards on updates/deletes:

  ```
  WHERE id = $1 AND ($2 = true OR company_id = $3)
  ```

  where `$2` is only true for `super_admin`.
- A super admin has `company_id = NULL`; record creation requires an explicit `company_id`, preventing accidental cross-tenant writes.
- Reports and dynamic workspace lists scope by the caller's company unless a super admin targets a company.
- Front-end menu visibility mirrors server checks but is **not** trusted — all enforcement is in the API.

5. Input handling / attack prevention

Threat	Mitigation
SQL injection	All queries use parameterised statements (`$1`, `$2`, …) via `pg`. Dynamic column names are taken only from whitelisted field lists, never raw input.
XSS	React escapes output by default; user text is stored/rendered as text.
CSRF	The API uses a bearer-token header (not cookies), so there is no ambient credential to exploit.
Brute force	Rate limiting and account lockout are recommended for production (see Deployment § hardening).
IDOR	Record reads/updates always verify ownership by company before returning data.
Overlong input	`normalizeField` truncates to field max (1 000 / 10 000 chars) and validates enums, ranges and formats (email, URL, date, numeric).
JSONB abuse	`service_snapshot` is constrained to a JSON object (`jsonb_typeof = 'object'`).

6. Data at rest

- **Secrets:** `DATABASE_URL` and `JWT_SECRET` live in `.env` (never committed — `.gitignore`). Rotate the JWT secret regularly.
- **Password hashes:** bcrypt with the app's configured cost; rehash policy on next login is recommended.
- **PII:** member names, phones, emails, addresses, medical records. Access to medical records is permission-gated (`spa_medical_records`).

- **Recommendation:** enable database-level encryption (e.g. managed provider disk encryption) and TLS between app and database (`sslmode=require`).

7. Audit logging

Every create/update/delete writes to `audit_logs` :

Column	Content
user_id / company_id	who, and which tenant
action	CREATE / UPDATE / DELETE
table_name	affected table (or `spa_management_records:<module>`)
record_id	affected row
old_values / new_values	jsonb before/after (DELETE logs the removed row)
ip_address / user_agent	request context
created_at	timestamp

- Audit writes are best-effort and must not roll back the primary transaction.
- The super admin can review all audit logs; a company admin can review their own company's logs.

8. Backup strategy

- **Database (Neon/PostgreSQL):** enable continuous backups (PITR) and take daily snapshots; retain ≥ 7 days, weekly for ≥ 4 weeks, monthly for ≥ 3 months.
- **Files:** only code and SQL are stored in the repo; no user data lives on disk.
- **Restore test:** restore to a staging database quarterly and verify reports render.

9. Deploy & environment hardening

- Run over **HTTPS** only; redirect HTTP.
- Set `NODE_ENV=production` ; never expose `.env` or the dev server publicly.
- Use a least-privilege DB role for the app (no `superuser`).
- Keep `npm audit` clean; pin dependency versions.
- Review `role_permissions` for every new role/resource before shipping.
- Follow the checklist in `DEPLOYMENT.md` §7.

10. Responsibilities

- **Operator:** secure workstations, use strong passwords, log out of shared terminals, never share admin accounts.

- **Admin:** grant roles minimally, disable leavers promptly, review audit logs regularly.
- **Platform owner:** manage companies/licenses, review cross-company access, rotate secrets.